

Міністерство освіти і науки України  
Національний університет «Львівська політехніка»  
Інститут комп'ютерних наук та інформаційних технологій  
Кафедра автоматизованих систем управління



## **Звіт**

до лабораторної роботи №6  
з дисципліни:

*“Паралельні обчислення і розподілені системи”*

на тему:

**“Розгортання у Kubernetes”**

Виконав: студент ОІ-31  
**Джебко Юрій**

Прийняв:  
Скорохода О. В.

Львів – 2026

## Лабораторна робота №6

### Розгортання у Kubernetes

**Мета роботи:** Розгортання мікросервісної системи у Kubernetes.

### Хід роботи

#### 1. Постановка задачі:

##### 1. Які сервіси розгортаються

У середовищі Kubernetes розгортається комплексна мікросервісна архітектура, що складається з п'яти компонентів. Для забезпечення мережевої взаємодії та балансування навантаження створюються відповідні об'єкти типу Service:

- Аплікаційні сервіси: service-warehouse, service-shipment та service-routing. Для цих сервісів налаштовано доступ із зовнішнього середовища за допомогою типу NodePort.
- Інфраструктурні сервіси: postgres та redis. Для них створено сервіси типу ClusterIP, які забезпечують безпечний ізольований доступ виключно всередині кластера через механізм внутрішнього DNS.

##### 2. Які Deployment-и створюються

Для декларативного управління життєвим циклом контейнерів створюється 5 об'єктів Deployment — по одному для кожного компонента системи:

- Інфраструктурні Deployment-и: базуються на офіційних образах із Docker Hub. Deployment для PostgreSQL додатково використовує об'єкт ConfigMap, через який у контейнер монтується SQL-скрипт для початкової ініціалізації трьох незалежних баз даних.
- Аплікаційні Deployment-и: використовують локально зібрані образи. У їхніх специфікаціях визначено ін'єкцію конфігурацій через змінні оточення, що дозволяє передавати параметри доступу до бази даних, кешу та URL-адреси для міжсервісної взаємодії.

##### 3. Яке масштабування реалізується

У рамках розгортання реалізується кілька механізмів масштабування та забезпечення високої доступності:

- Декларативне горизонтальне масштабування: базовий маніфест для service-warehouse одразу налаштовується на запуск кількох екземплярів.
- Динамічне масштабування: виконується оперативне збільшення кількості реплік service-shipment (від 1 до 3) у процесі роботи кластера для обробки

більшого навантаження. Трафік між ними розподіляється вбудованим балансувальником Kubernetes.

- Автоматичне відновлення: система постійно контролює відповідність фактичного стану до бажаного. У разі примусового видалення або збою Pod-a, контролер автоматично створює новий екземпляр без втручання адміністратора.
- Оновлення без простою: застосовується стратегія поступового оновлення версії мікросервісу. Kubernetes по черзі зупиняє старі Pod-и та запускає нові, гарантуючи нульовий час простою під час розгортання нових релізів.

## 2. Проектування:

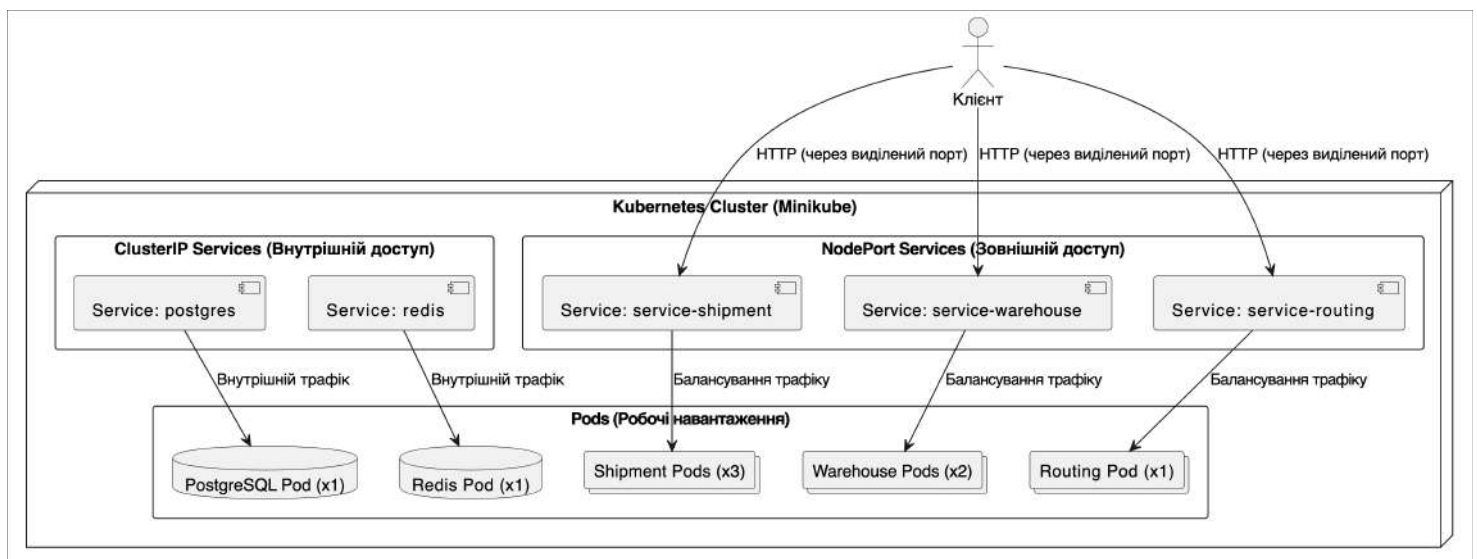


Рис. 1. Схема кластера.

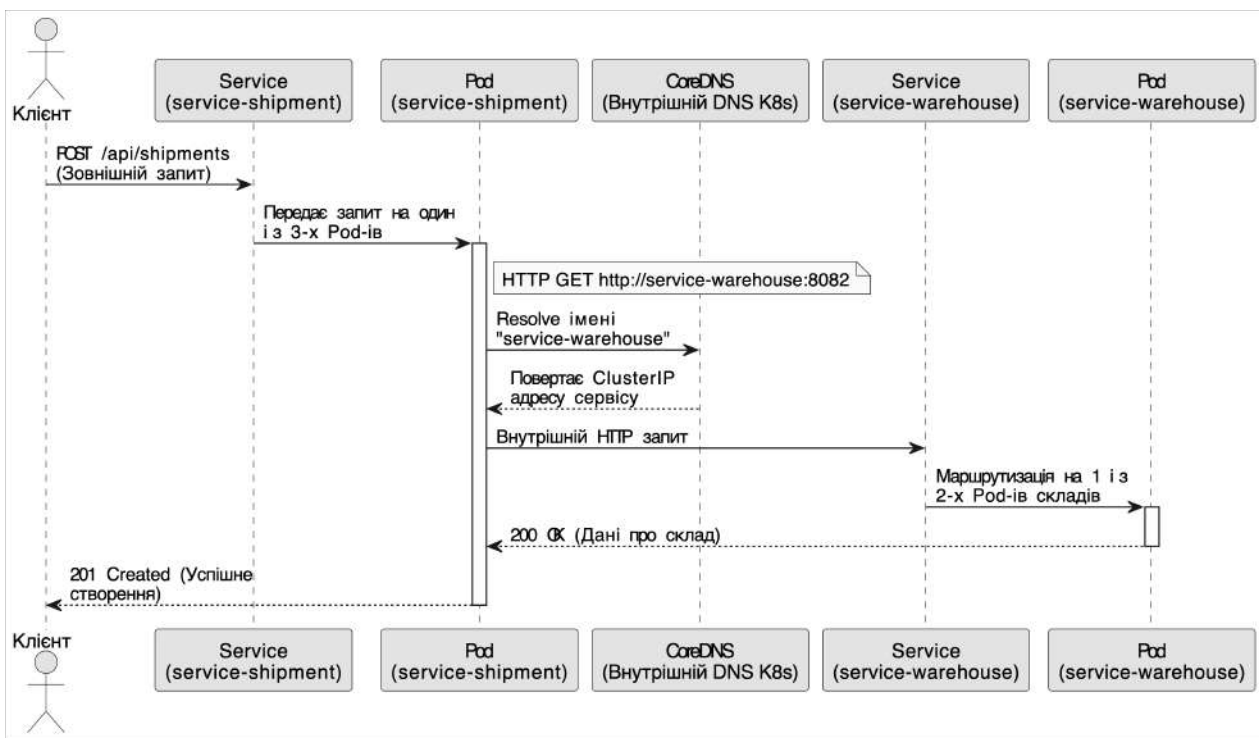


Рис. 2. Взаємодія компонентів.

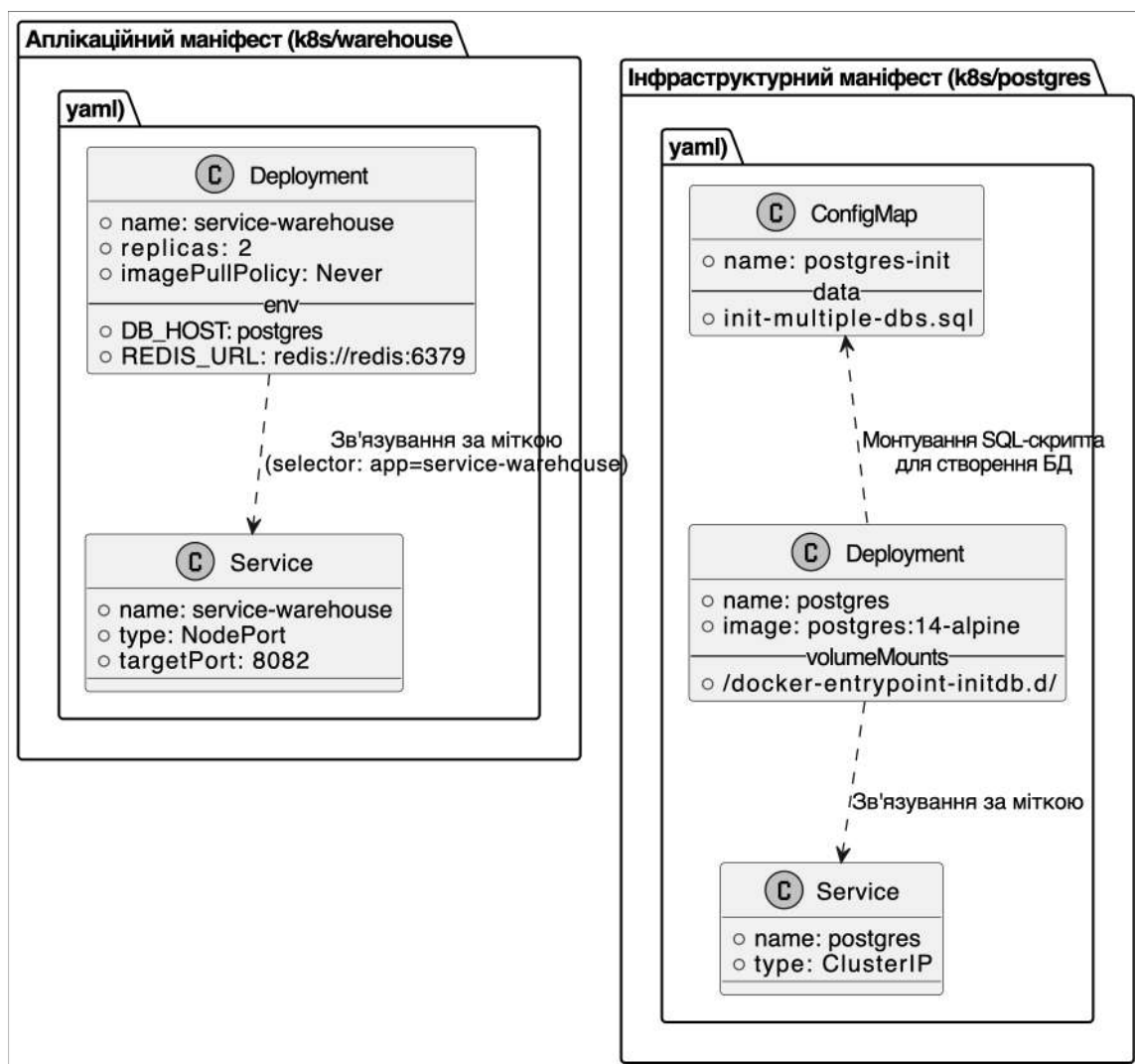


Рис. 3. Структура YAML-файлів.

### 3. Реалізація:

Приклад yaml-файлу для мікросервіса:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: service-warehouse
spec:
  replicas: 2
  selector:
    matchLabels:
      app: service-warehouse
  template:
    metadata:
      labels:
        app: service-warehouse
    spec:
      containers:
      - name: service-warehouse
        image: logistics-warehouse:latest
        imagePullPolicy: Never
        ports:
        - containerPort: 8082
        env:
        - name: PORT
          value: "8082"
        - name: DB_HOST
          value: postgres
        - name: DB_PORT
          value: "5432"
        - name: DB_NAME
          value: warehouse_db
        - name: DB_USER
          value: user
        - name: DB_PASSWORD
          value: password
        - name: REDIS_URL
          value: redis://redis:6379
---
apiVersion: v1
kind: Service
metadata:
  name: service-warehouse
spec:
  type: NodePort
  selector:
    app: service-warehouse
  ports:
  - protocol: TCP
    port: 8082
    targetPort: 8082
```

Приклад yaml-файлу для бази даних:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-init
data:
  init-multiple-dbs.sql: |
    CREATE DATABASE shipment_db;
    CREATE DATABASE warehouse_db;
```

```

CREATE DATABASE routing_db;
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:14-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              value: user
            - name: POSTGRES_PASSWORD
              value: password
          volumeMounts:
            - name: init-script
              mountPath: /docker-entrypoint-initdb.d/init-multiple-dbs.sql
              subPath: init-multiple-dbs.sql
      volumes:
        - name: init-script
          configMap:
            name: postgres-init
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  selector:
    app: postgres
  ports:
    - protocol: TCP
      port: 5432

```

Приклад yaml-файлу для кешування:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis

```

```

      image: redis:alpine
      ports:
      - containerPort: 6379
----
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  selector:
    app: redis
  ports:
  - protocol: TCP
    port: 6379

```

Команди kubectl:

```

(base) user@MacBook-Air-User microservices % kubectl get pods -w
NAME                                READY   STATUS             RESTARTS   AGE
postgres-76f7d7c775-wjp4l          0/1     ContainerCreating   0          20s
redis-68bbc94689-sgzjp             0/1     ContainerCreating   0          20s
service-routing-fc875b8b9-7k4vt    1/1     Running             0          20s
service-shipment-57df7fb894-qv65z  1/1     Running             0          19s
service-warehouse-854bf4755b-5dw5p  1/1     Running             0          19s
service-warehouse-854bf4755b-h8dbr  1/1     Running             0          19s
postgres-76f7d7c775-wjp4l          1/1     Running             0          49s
redis-68bbc94689-sgzjp             1/1     Running             0          61s

```

Рис. 4. Процес підняття всіх Pod-ів

## 4. Тестування:

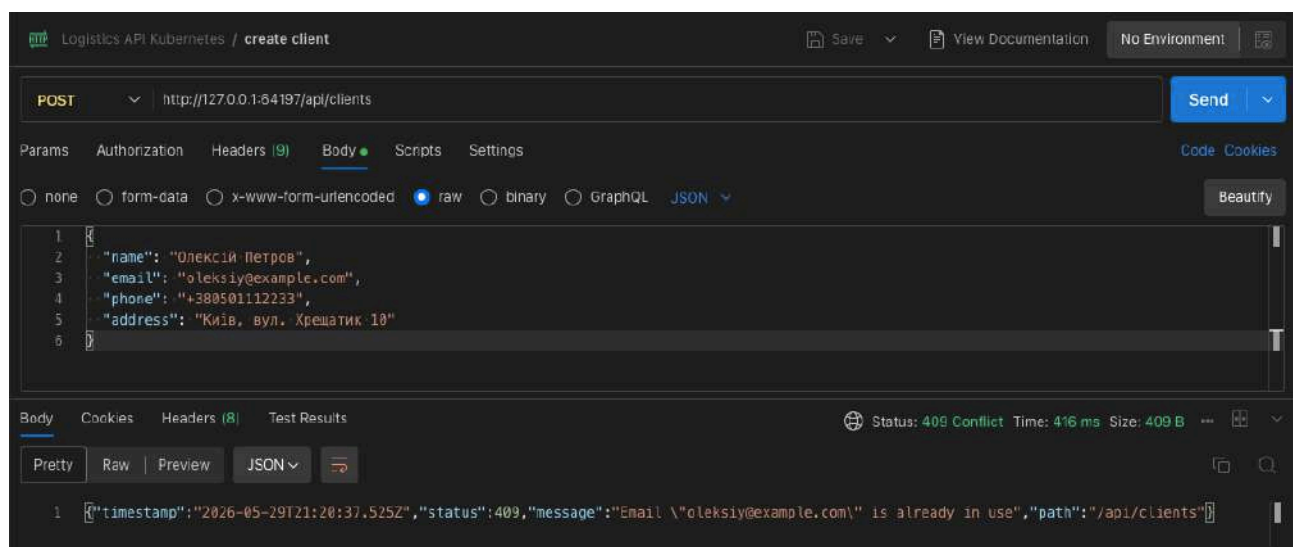


Рис. 5. Перевірка коректності запитів. Все працює!

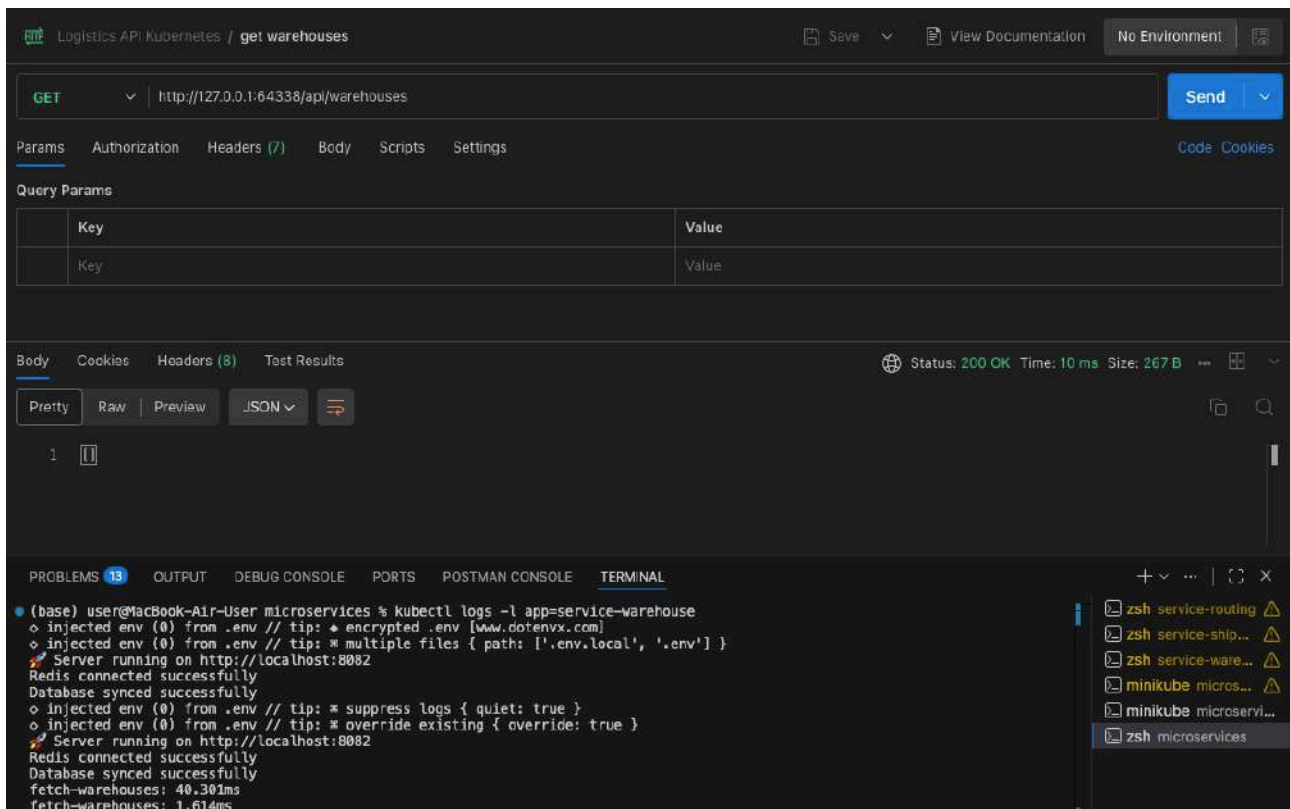


Рис. 6. Перевірка кешування. Redis під'єднався та час запиту зменшився.

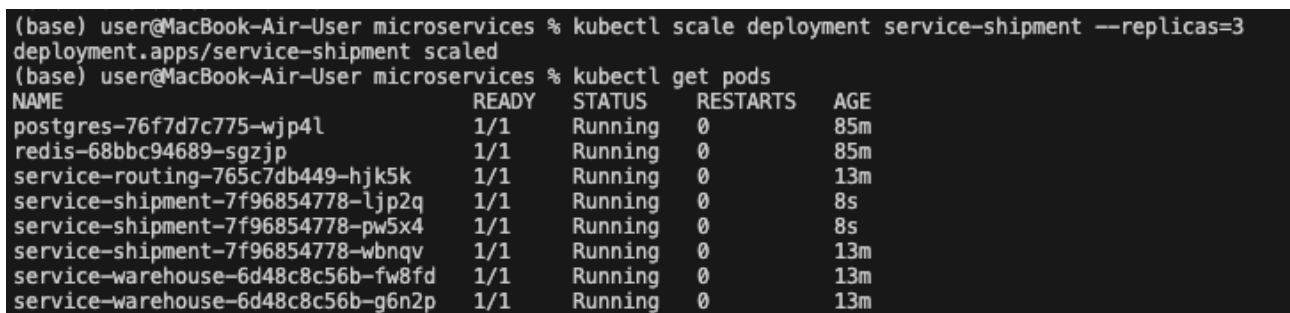


Рис. 7. Замість одного service-shipment тепер працюють три Pod-и

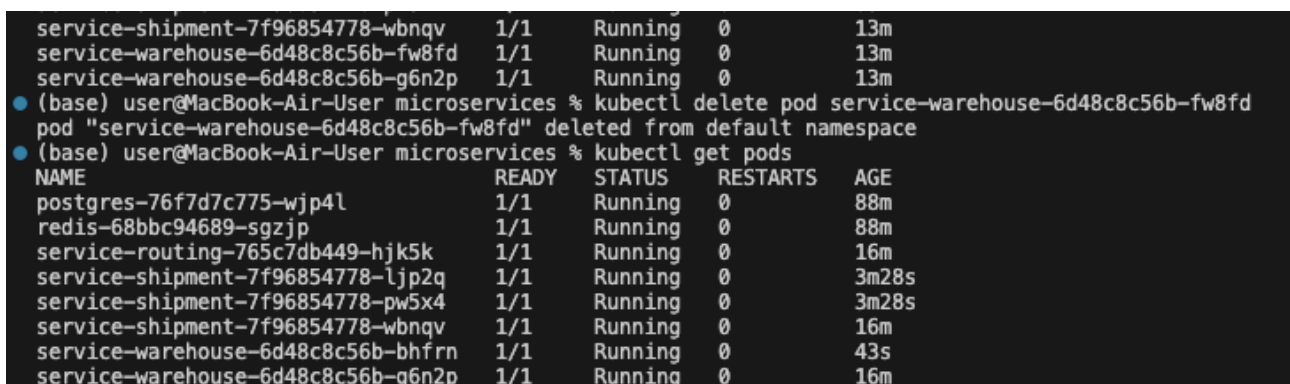


Рис. 8. Поведінка при зупинці Pod. Старий pod service-warehouse-6d48c8c56b-fw8fd замінився на новий.



```
(base) user@MacBook-Air-User microservices % kubectl set image deployment/service-warehouse service-warehouse=logistics-warehouse:v2
deployment.apps/service-warehouse image updated
(base) user@MacBook-Air-User microservices % kubectl rollout status deployment/service-warehouse
deployment "service-warehouse" successfully rolled out
```

Рис. 9. Оновлення без зупинки (Rolling Update)

## 5. Аналіз:

### 1. Порівняння з Docker Compose

Хоча обидва інструменти використовують контейнеризацію, вони вирішують завдання різного масштабу:

- Середовище виконання: Docker Compose призначений для розгортання багатоконтейнерних застосунків на одному фізичному вузлі (сервері або комп'ютері розробника). Kubernetes — це розподілена система, здатна керувати контейнерами на сотнях серверів, об'єднуючи їх у єдиний пул ресурсів.
- Керування станом: У Docker Compose підхід здебільшого імперативний (запуск команд up/down). K8s використовує суворо декларативний підхід: інженер описує бажаний стан системи (наприклад, «потрібно 3 репліки»), а кластер безперервно працює над тим, щоб фактичний стан відповідав заданому.
- Мережа та балансування: Docker Compose створює просту локальну мережу, де контейнери бачать один одного за іменами. K8s надає повноцінний внутрішній DNS, розширене балансування трафіку через об'єкти Service та розділення доступів.

### 2. Переваги оркестрації

Перехід від простої контейнеризації до оркестрації за допомогою Kubernetes забезпечив системі низку критично важливих архітектурних переваг:

- Висока доступність та самовідновлення: Система автоматично виявляє завислі або впавші контейнери і миттєво перестворює їх на здорових вузлах, гарантуючи безперервну роботу застосунку без втручання адміністратора.
- Еластичне масштабування: Можливість горизонтально масштабувати окремі мікросервіси (наприклад, збільшити кількість service-shipment під час пікових навантажень) однією командою, з автоматичним розподілом трафіку між новими репліками.

- Оновлення без даунтайму: Kubernetes дозволяє розгортати нові версії мікросервісів поступово, замінюючи старі контейнери новими по черзі. Це забезпечує нульовий час простою під час релізів.

### 3. Доцільність використання

Використання Kubernetes є абсолютно доцільним для production-середовищ складних мікросервісних архітектур, до яких висуваються суворі вимоги щодо надійності, відмовостійкості та здатності витримувати змінні навантаження. Для розробленої логістичної системи K8s забезпечив надійну ізоляцію інфраструктури (СУБД, кеш) від зовнішнього світу та створив фундамент для подальшого зростання проєкту.

Водночас, використання Kubernetes не є доцільним для локальної розробки або розгортання простих монолітних застосунків з низьким навантаженням через високий поріг входження, складність налаштування маніфестів та значне споживання системних ресурсів самими компонентами кластера. Для таких завдань Docker Compose залишається більш оптимальним інструментом.

### **Висновок:**

У ході виконання роботи було успішно здійснено міграцію логістичної мікросервісної системи з локального середовища Docker Compose у повноцінний оркестратор контейнерів Kubernetes.

Завдяки декларативному підходу через YAML-маніфести було налаштовано автоматичне розгортання та керування як інфраструктурними вузлами (PostgreSQL, Redis), так і трьома незалежними застосунками. Налаштування об'єктів Service забезпечило надійну мережеву ізоляцію баз даних та ефективне балансування зовнішнього і внутрішнього трафіку.

Практичне тестування кластера підтвердило ключові переваги оркестрації. Система успішно продемонструвала здатність до динамічного горизонтального масштабування, миттєве автоматичне відновлення після імітації критичного збою контейнера, а також можливість безперебійного оновлення версій мікросервісів із нульовим часом простою. У результаті архітектуру проєкту виведено на новий рівень надійності, що повністю відповідає стандартам сучасних відмовостійких Production-рішень.